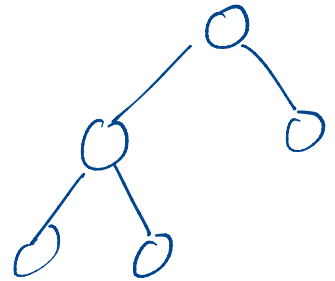
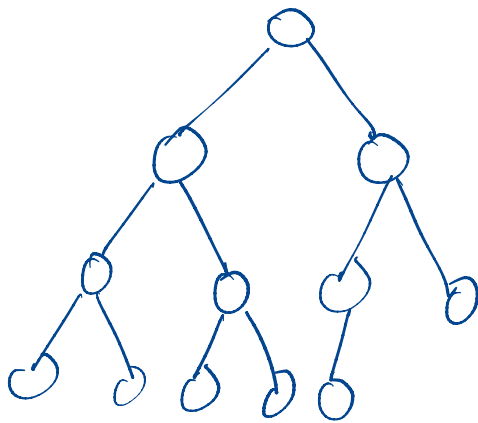


AVL Trees

A full binary tree is one in which all nodes have either two or no children.



In a complete binary tree every level, except possibly the last level, is completely filled, and all nodes in the last level are as far left as possible.



In a balanced tree, the left & the right subtrees of every node differ in height by no more than 1.

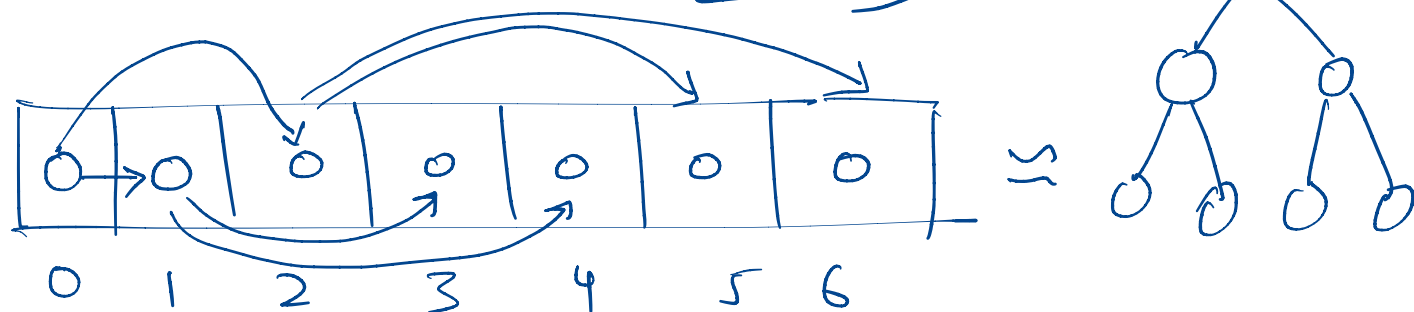
Defn.: Height of a subtree is given by the number of edges from the topmost node to the farthest node in the subtree

[if subtree is empty, then height = -1] CHECK!!

— side note —

Array representation of a binary tree:

- Store in a breadth-first order
- A node with index i will have its left child stored at index $2i+1$, and right child stored at index $2i+2$
- root is at index 0
- The parent of a node, if exists, can be found at index $\left\lfloor \frac{(i-1)}{2} \right\rfloor$



→ No space wasted if tree is complete binary tree

- Problems/issues :
- ① space wasted for imbalanced / skewed trees
 - ② Expansion is costly
-

AVL trees

→ Set operations are fast if the height is small

→ Goal is to minimize height

→ maintain logarithmic height $\sim \log_2 n$
 $n = \# \text{ of nodes}$

→ Is a self balancing tree

→ Proposed by G.M. Adelson - Velskii
and E.M. Landis
in 1962.

→ On an average search takes

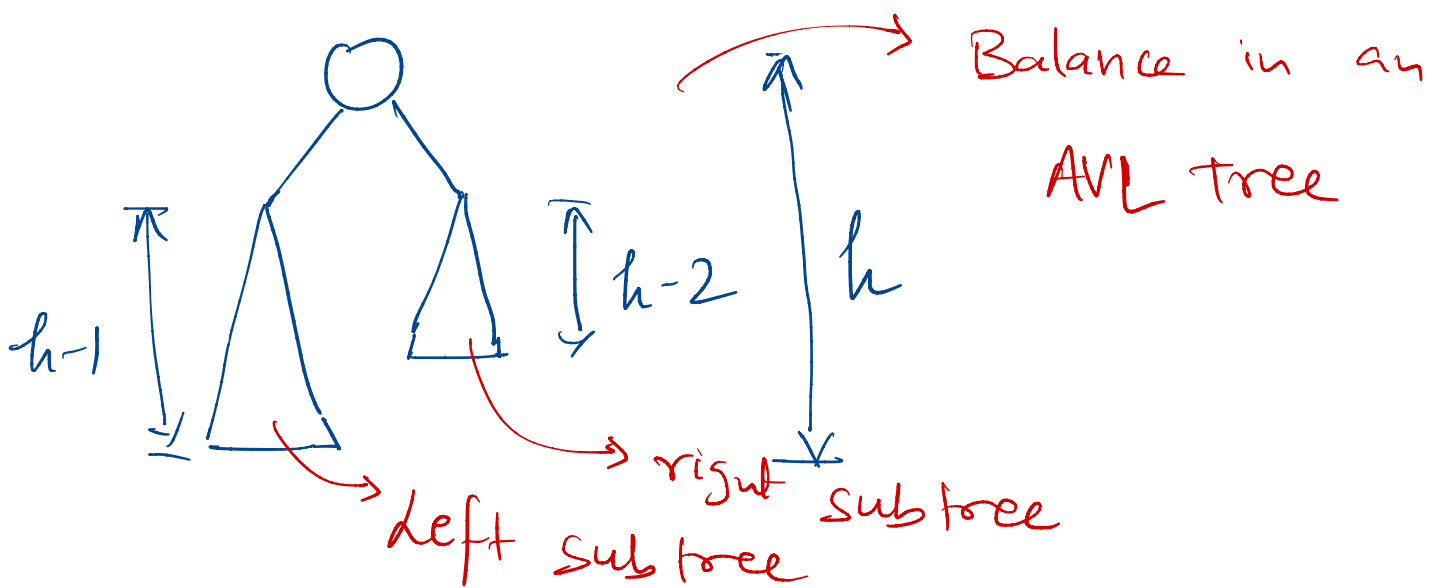
$1.44 \log_2 n$ comparisons!

(Pretty close to $\log_2 n$)

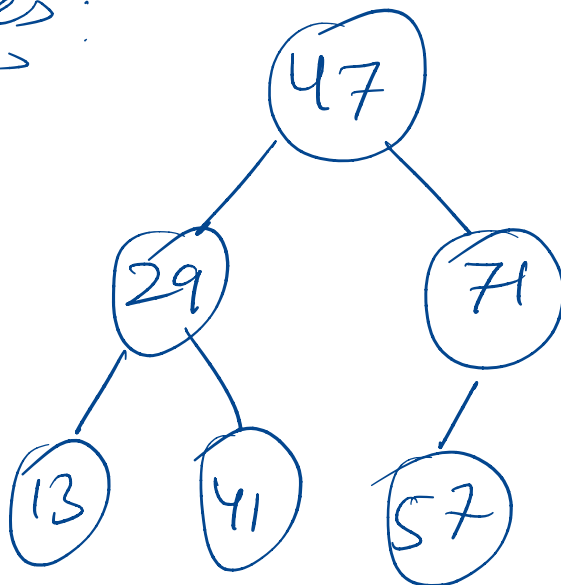
Defn. : An AVL tree is a BST with Properties :

1. The subtrees of every node differ in height by at most one.

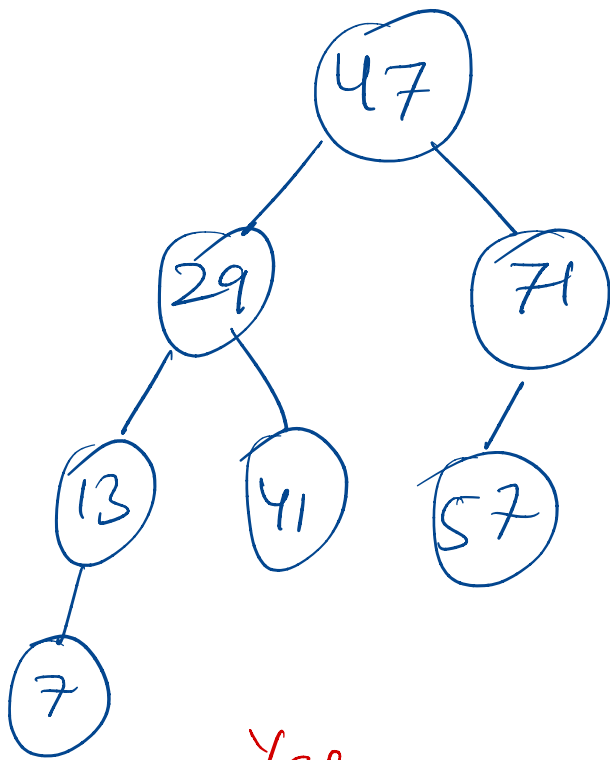
$\Rightarrow$ Every subtree is an AVL tree



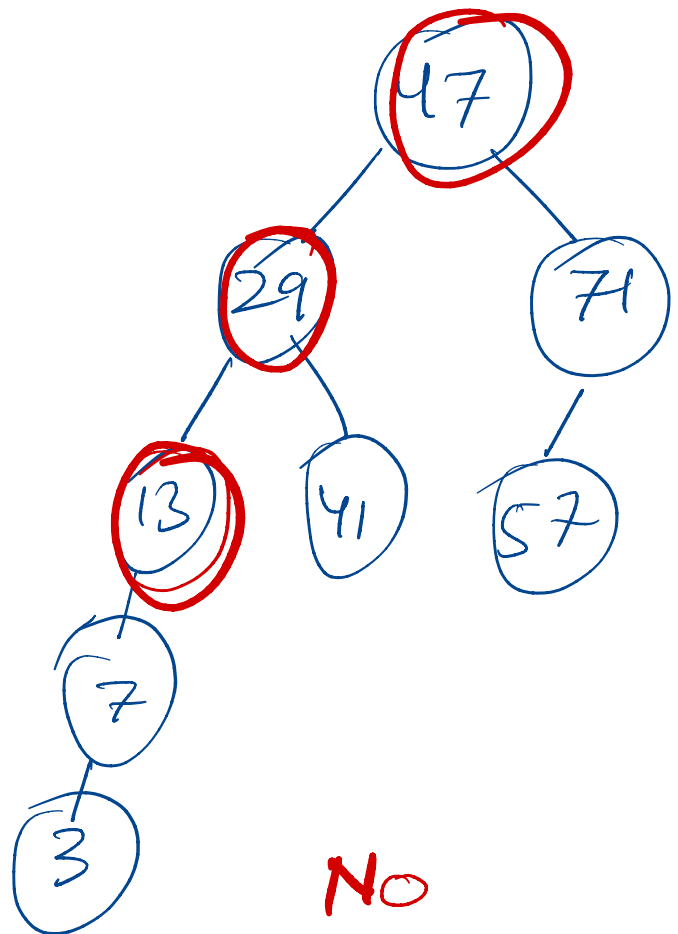
Examples :



Yes

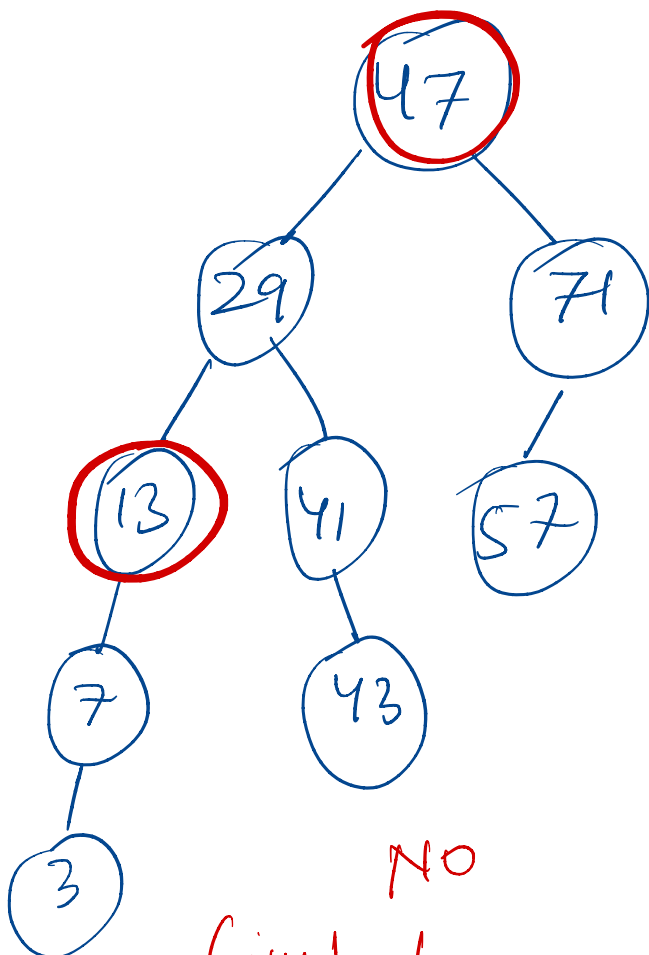


Yes



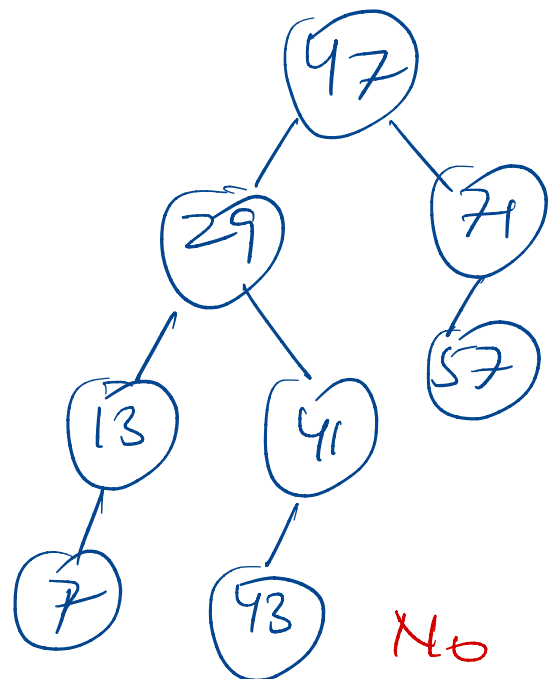
No

(imbalance at 13, 29, & 47)



No

(imbalance at 13, 47)



No

(Not a BST)

An AVL tree needs to be rebalanced after insertion or deletion.

→ rebalancing is performed via rotations (single or double)

→ Insertion & deletion are ~~to~~ handled in the same way as in a BST

→ We keep a value at each node to denote balance of a node

Balance Factor (BF) : of a node X is defined as the height difference

$$BF(X) = \text{Height}(\underbrace{X.\text{right}}_{\text{right child}}) - \text{Height}(\underbrace{X.\text{left}}_{\text{left child}})$$

$$\Rightarrow \underline{BF(x)} \in \{-1, 0, 1\}$$

A node is :

Left heavy if $BF(x) < 0$

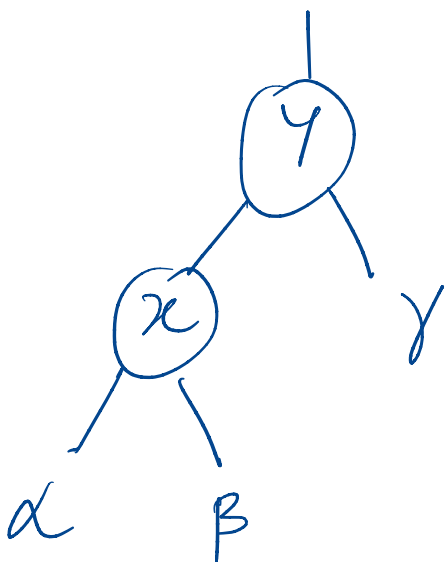
Right heavy if $BF(x) > 0$

Balanced if $BF(x) = 0$

Rotations

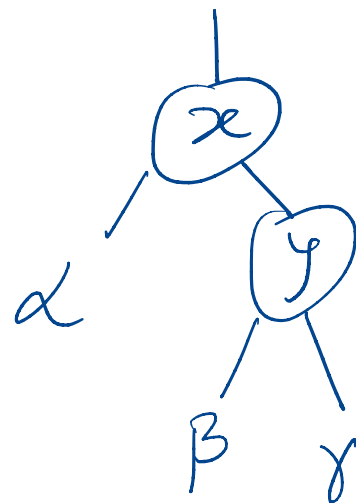
left right

local operation that
Preserves the binary-search-tree
Property.

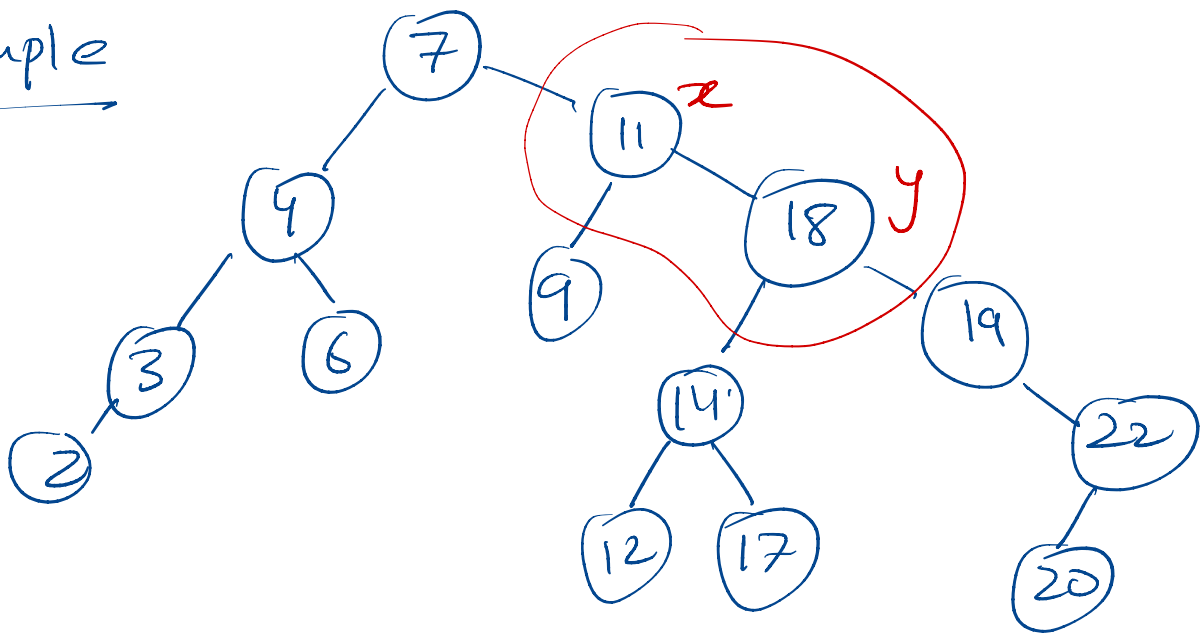


Right-rotate(T, y)

Left-rotate(T, x)



Example



↓ left rotate (x)

